

Chapter 6: Markov Chain Monte Carlo Methods

Lectures on Monte Carlo Theory

Leskelä & Vihola

Chapter Overview

- 1 6.1 Introduction
- 2 6.2 Markov Chains
- 3 6.3 Constructing a Markov Chain with Prescribed Stationary Distribution
 - 6.3.1 Metropolis and Metropolis-Hastings
 - 6.3.2 Gibbs Sampler
 - Graph Coloring
 - Hard-core Model
 - Ising Model
- 4 6.4 Perfect Simulation: Sampling Exactly from π
 - 6.4.1 CFTP Algorithm
- 5 6.5 Approximate Counting
 - 6.5.1 Independent Sets — FPRAS
 - 6.5.2 Graph Colorings and Knapsack
- 6 6.6 Computational Biology: Motif Finding
- 7 6.7 Estimating $I = \mathbb{E}f(X)$, $X \sim \pi$
 - 6.7.1 Ergodic Theorem and CLT
- 8 6.8 Stable Simulation Scheme
- 9 6.9 Bibliographical Notes
- 10 6.10 Exercises (Selected)

6.1 Introduction: Motivation

Goal: Estimate $I = \mathbb{E}f(Y)$ or $I = \mathbb{E}Y$ where $Y \sim \pi$.

Standard Monte Carlo: Generate independent samples $Y_1, \dots, Y_R \sim \pi$, use

$$\hat{Y}_R = \frac{1}{R} \sum_{i=1}^R f(Y_i) \approx I.$$

The Problem

In many applications π is **intractable**:

- The state space $\mathcal{S}$ is exponentially large (e.g. $|\mathcal{S}| = 2^d$).
- Direct sampling requires a normalizing constant z_λ that is unknown.
- Example: $\pi_\sigma = \frac{1}{z_\lambda} e^{\lambda f(\sigma)}$ (Gibbs/Boltzmann distribution).

MCMC Solution

Construct a **Markov chain** (X_n) whose stationary distribution is π . Then

$$\hat{Y}_R = \frac{1}{R} \sum_{i=1}^R f(X_i) \xrightarrow{R \rightarrow \infty} I \quad (\text{ergodic theorem}).$$

6.1 Introduction: Running Example

Permutations and fixed points. Let $Y \sim \pi_n = \text{uniform on } S_n$ (all permutations of n elements). Let

$$f(\sigma) = \#\{i : \sigma(i) = i\} \quad (\text{number of fixed points}).$$

Goal: Estimate $I = \mathbb{E}f(Y)$.

Easy case (π uniform): generate random permutations directly.

Hard case (π non-uniform): suppose

$$\pi_\sigma = \frac{1}{z_\lambda} e^{\lambda f(\sigma)}, \quad z_\lambda = \sum_{\sigma} e^{\lambda f(\sigma)}.$$

Computing z_λ is **infeasible** for large n .

MCMC approach:

- 1 Build a Markov chain on S_n with stationary π .
- 2 Run the chain for R steps.
- 3 Average $f(X_j)$ to estimate I .

6.2 Markov Chains: Definitions

Markov Chain

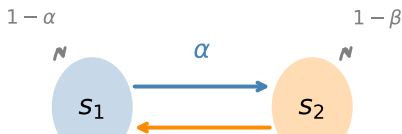
A sequence $(X_n)_{n \geq 0}$ taking values in $\mathcal{S} = \{s_1, \dots, s_N\}$ is a Markov chain with **transition matrix** $\mathbf{P} = (p_{ss'})$ if

$$\mathbb{P}(X_{n+1} = s' \mid X_0, \dots, X_n) = p_{X_n s'}, \quad \text{for all } n \geq 0.$$

Key definitions:

- **Stationary distribution** π : $\pi_s = \sum_{s' \in \mathcal{S}} \pi_{s'} p_{s's}$ for all s , i.e. $\pi \mathbf{P} = \pi$.
- **Distribution at step n** : $\mu^n = \mu \mathbf{P}^n$.
- **Irreducibility**: all states communicate ($p_{ss'}^{(n_0)} > 0$ for some n_0).
- **Aperiodicity**: every state has period 1.
- **Ergodicity (Regularity)**: $\mathbf{P}$ is ergodic if $\exists n_0$ s.t. $\mathbf{P}^{n_0} > \mathbf{0}$.

Two-state Markov chain



$$\text{Two-state chain: } \mathbf{P} = \begin{pmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{pmatrix}$$

$$\text{Stationary: } \pi = \left(\frac{\beta}{\alpha + \beta}, \frac{\alpha}{\alpha + \beta} \right)$$

6.2 Ergodic Theorem and Convergence

Proposition 6.2.2 (Ergodic Theorem)

If $\mathbf{P}$ is **regular** (ergodic), then there exists a unique stationary distribution π with all entries strictly positive, and

$$\mathbf{P}^n \rightarrow \mathbf{\Pi}, \quad \text{as } n \rightarrow \infty,$$

where $\mathbf{\Pi}$ is the matrix with each row equal to π .

Corollary 6.2.3

For any initial distribution μ :

$$\mu \mathbf{P}^n \rightarrow \pi \quad \text{as } n \rightarrow \infty.$$

Total variation distance:

$$d_{\text{TV}}(\mu, \nu) = \frac{1}{2} \sum_{s \in \mathcal{S}} |\mu_s - \nu_s|.$$

Mixing time:

$$\tau(\varepsilon) = \min_{n \in \mathbb{N}} \{d_{\text{TV}}(\mu \mathbf{P}^n, \pi) \leq \varepsilon\}.$$

6.2 Update Function and Detailed Balance

Proposition 6.2.4 (Update Function)

Given $\mathbf{P}$ and initial distribution μ , there exist functions $\varphi : \mathcal{S} \times [0, 1] \rightarrow \mathcal{S}$ and $\psi : [0, 1] \rightarrow \mathcal{S}$ such that

$$X_0 = \psi(U_0), \quad X_{n+1} = \varphi(X_n, U_{n+1}),$$

where $U_0, U_1, \dots$ are i.i.d. $\mathcal{U}[0, 1]$.

Proposition 6.2.5 (Detailed Balance Condition — DBC)

If positive weights $(w_s)_{s \in \mathcal{S}}$ satisfy

$$w_s p_{ss'} = w_{s'} p_{s's}, \quad \forall s, s' \in \mathcal{S},$$

then $\pi_s = w_s / \sum_{s'} w_{s'}$ is a stationary distribution.

The DBC is equivalent to **reversibility**: $(X_0, X_1, \dots, X_n) \stackrel{d}{=} (X_n, X_{n-1}, \dots, X_0)$ when $X_0 \sim \pi$.

6.2 Random Walks on Graphs

Example 6.2.7 (Simple Random Walk on Graph G)

For a graph $G = (V, E)$, the transition matrix $\mathbf{Q} = (q_{vv'})$ is

$$q_{vv'} = \begin{cases} \frac{1}{\deg(v)}, & \text{if } v' \in \mathcal{N}(v), \\ 0, & \text{otherwise.} \end{cases}$$

Setting $w_v = \deg(v)$ satisfies DBC, so the stationary distribution is

$$\pi_v = \frac{\deg(v)}{2|E|}, \quad v \in V.$$

Self-loop modification (for aperiodicity): with probability $\beta \in (0, 1)$, stay at current vertex:

$$q_{vv'} = \begin{cases} \beta, & v = v', \\ \frac{1-\beta}{\deg(v)}, & v' \in \mathcal{N}(v). \end{cases}$$

This does not change the stationary distribution.

6.3 MCMC: The Big Idea

Problem

Given a **target distribution** π on a large state space $\mathcal{S} = \{s_1, \dots, s_N\}$, construct a Markov chain (Y_n) with stationary distribution π that is easy to simulate.

Goals of MCMC:

- (A1) Sample from a distribution “close” to π (and in some cases, exactly from π).
- (A2) Approximate $\pi(A) = \sum_{s \in A} \pi_s$ for specific subsets $A \subseteq \mathcal{S}$.

Key insight: We only need to compute *ratios* $\pi_{s'}/\pi_s$ — the normalizing constant cancels.

Two main approaches:

- **Metropolis-Hastings:** propose a move, accept/reject based on ratio.
- **Gibbs sampler:** update one coordinate at a time from its full conditional.

6.3.1 Metropolis Algorithm

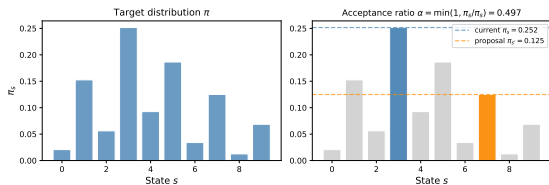
Setup

Let $\mathbf{Q} = (q_{ss'})$ be a **symmetric**, regular transition matrix (the *proposal* matrix). Define the acceptance probability:

$$\alpha = \min\left(1, \frac{\pi_{s'}}{\pi_s}\right).$$

Algorithm 42 (Metropolis):

- 1 Current state: $Y_k = s$.
- 2 Propose $X \sim \mathbf{q}_s$.
- 3 Set $\alpha = \min(1, \pi_X/\pi_s)$.
- 4 Generate $U \sim \mathcal{U}[0, 1]$.
- 5 If $U \leq \alpha$: $Y_{k+1} = X$.
- 6 Else: $Y_{k+1} = Y_k$.



Proposition 6.3.1

If $\mathbf{Q}$ is symmetric and regular, then $\mathbf{P}$ is regular with stationary distribution π .

6.3.1 Metropolis-Hastings Algorithm

Extension: Drop the symmetry assumption on $\mathbf{Q}$.

Algorithm 43 (Metropolis-Hastings)

- 1 Current state: $Y_k = s$.
- 2 Propose $X \sim \mathbf{q}_s$.
- 3 Set $\alpha = \min\left(1, \frac{\pi_X q_{Xs}}{\pi_s q_{sX}}\right)$.
- 4 If $U \leq \alpha$: accept $Y_{k+1} = X$; else $Y_{k+1} = Y_k$.

Proposition 6.3.2 (Metropolis-Hastings)

Let $\mathbf{Q}$ be regular with $q_{ss'} > 0 \Leftrightarrow q_{s's} > 0$. Then $\mathbf{P}$ with

$$p_{ss'} = q_{ss'} \alpha_{ss'}, \quad s \neq s', \quad \alpha_{ss'} = \min\left(1, \frac{\pi_{s'} q_{s's}}{\pi_s q_{ss'}}\right),$$

is a regular matrix with stationary distribution π .

Key: Only ratios $\pi_{s'}/\pi_s$ are needed — normalizing constant cancels.

6.3.1 Metropolis-Hastings: Python Implementation

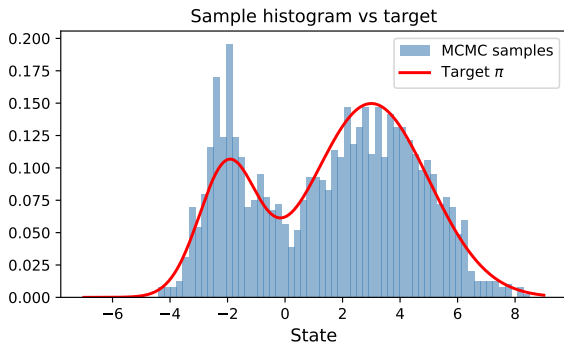
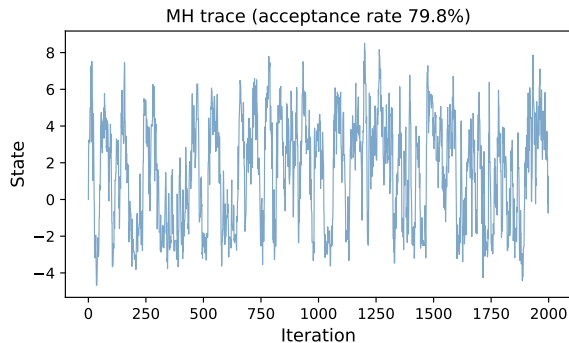
```
import numpy as np

def metropolis_hastings(log_pi, proposal, x0, n_steps):
    """
    Generic Metropolis-Hastings sampler.
    log_pi : callable, log of (unnormalized) target density
    proposal: callable, proposal(x) -> (x_new, log_q_ratio)
              where log_q_ratio = log q(x_new->x) - log q(x->x_new)
    x0      : initial state
    n_steps : number of steps
    """
    samples = [x0]
    x = x0
    n_accept = 0
    for _ in range(n_steps):
        x_new, log_q_ratio = proposal(x)
        # MH acceptance ratio
        log_alpha = (log_pi(x_new) - log_pi(x)) + log_q_ratio
        if np.log(np.random.rand()) < log_alpha:
            x = x_new
            n_accept += 1
        samples.append(x)
    print(f"Acceptance rate: {n_accept/n_steps:.1%}")
    return np.array(samples)
```

Example: target = mixture of Gaussians, symmetric random walk proposal

```
def log_pi_mix(x):
```

6.3.1 MH Trace and Convergence



Observations:

- After a **burn-in** period the chain mixes and samples from π .
- The proposal step size σ controls the acceptance rate (target $\approx 20\text{--}50\%$).
- The sample histogram converges to the target density.

6.3.2 Gibbs Sampler: Setup

Setting: State space $\mathcal{S} = C^V$ where V is a set of vertices (e.g. graph nodes) and $C = \{1, \dots, r\}$ is a finite set of values.

Full Conditional Distribution

For a configuration $\xi \in \mathcal{S}$ and vertex $v \in V$, define

$$p_{\xi,v}(c) = \mathbb{P}(Z(v) = c \mid Z_{-v} = \xi_{-v}), \quad c \in C,$$

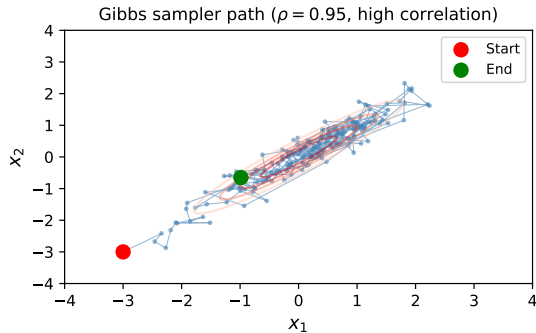
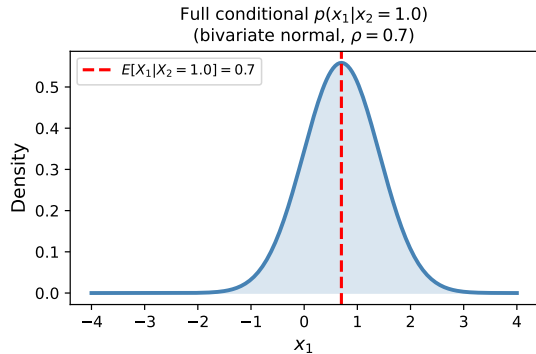
where $Z \sim \pi$ and ξ_{-v} is ξ with the v -th component removed.

Gibbs Sampler (one step)

- 1 Current configuration: ξ .
- 2 Pick vertex $v \in V$ uniformly at random.
- 3 Sample $c \sim p_{\xi,v}(\cdot)$ (the **full conditional**).
- 4 Set $\xi' = \xi(v, c)$ (replace ξ_v with c).

Key property: The Gibbs sampler satisfies DBC and has stationary distribution π .

6.3.2 Gibbs Sampler: Full Conditional Illustration



Left: Full conditional $p(x_1|x_2 = 1)$ for a bivariate normal with $\rho = 0.7$.

Right: Gibbs sampler path for high correlation ($\rho = 0.95$) — characteristic zigzag pattern showing slower mixing than MH.

6.3.2 Gibbs Sampler: Python Implementation

```
import numpy as np

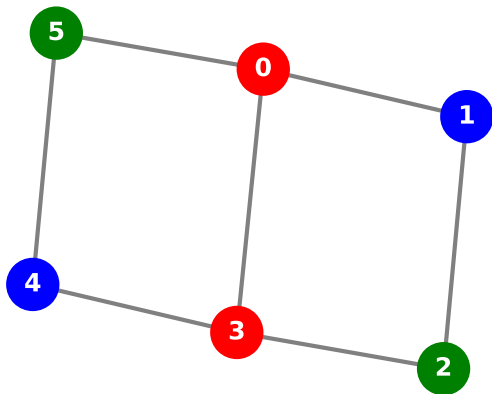
def gibbs_sampler(pi_dict, state_space, vertices, n_steps, xi0=None):
    """
    Generic Gibbs sampler on  $S = C^V$ .
    pi_dict : dict mapping configurations (as tuples) to unnormalized probabilities
    vertices : list of vertex labels
    n_steps : number of Gibbs steps
    """
    import random
    xi = dict(xi0) if xi0 else {v: state_space[0] for v in vertices}
    samples = [tuple(xi[v] for v in vertices)]

    for _ in range(n_steps):
        v = random.choice(vertices)    # pick vertex uniformly
        # Compute full conditional
        weights = []
        for c in state_space:
            xi_test = {**xi, v: c}
            weights.append(pi_dict.get(tuple(xi_test[u] for u in vertices), 0))
        total = sum(weights)
        probs = [w/total for w in weights]
        # Sample from full conditional
        xi[v] = np.random.choice(state_space, p=probs)
        samples.append(tuple(xi[v] for v in vertices))
    return samples
```


6.3.2.1 Graph Coloring with Gibbs Sampler

Setup: Graph $G = (V, E)$, $r \geq 2$ colors. A **proper r -coloring** assigns colors from $C = \{1, \dots, r\}$ to vertices such that no two adjacent vertices share a color.

Proper 3-coloring of a graph



Uniform Distribution on Proper Colorings

$$\pi_{\xi} = \frac{1}{z_G} \mathbf{1}(\xi \in \Lambda),$$

where Λ = set of all proper colorings, $z_G = |\Lambda|$.

Gibbs step:

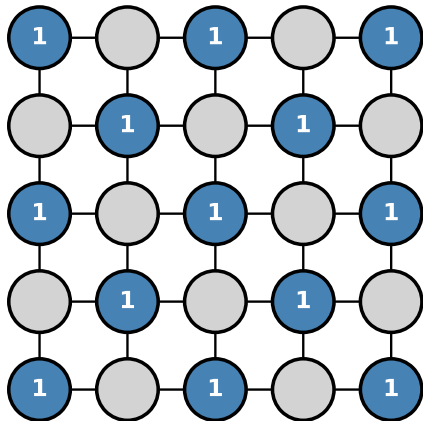
- 1 Pick v uniformly.
- 2 Let $F(v, \xi) =$ colors of v 's neighbors.
- 3 Pick c uniformly from $C \setminus F(v, \xi)$.
- 4 Set $\xi_v \leftarrow c$.

Theorem (Jerrum '95): If $r \geq 2\Delta + 1$ (where $\Delta = \max \deg$), the Gibbs sampler is **rapidly mixing**

6.3.2.2 Hard-core Model

Setup: Graph $G = (V, E)$, $\lambda > 0$. Each vertex is either *occupied* (1) or *empty* (0). An **independent set** (hard-core configuration) has no two adjacent vertices occupied.

Hard-core model: independent set on grid



Hard-core Distribution

$$\pi_{\xi} = \frac{\lambda^{\sum_v \xi(v)}}{Z_{\lambda}} \mathbf{1}(\xi \in \Lambda),$$

Λ = all independent sets.

Full conditional (Eq. 6.3.13):

$$p_{\xi,v}(1) = \frac{\lambda}{1 + \lambda} \mathbf{1}(\text{all nbrs} = 0)$$

6.3.2.4 Ising Model

Setup: Graph $G = (V, E)$, $\beta \in \mathbb{R}$. Each vertex has spin $\xi(v) \in \{-1, +1\}$.

Ising Distribution

$$\pi_{\xi} = \frac{1}{z(\beta)} \exp \left(\beta \sum_{\{u,v\} \in E} \xi(u) \xi(v) \right).$$

- $\beta > 0$: ferromagnet (spins align with neighbors).
- $\beta < 0$: anti-ferromagnet (spins oppose neighbors).

Gibbs update function:

$$\varphi(\xi, v, u) = \begin{cases} \xi(v \rightarrow -1) & \text{if } u \in [0, \alpha(\xi, v)), \\ \xi(v \rightarrow +1) & \text{if } u \in [\alpha(\xi, v), 1], \end{cases}$$

where $\alpha(\xi, v) = \mathbb{P}(Z(v) = -1 \mid Z_{-v} = \xi_{-v})$.

Ising model sample ($\beta = 0.45$, 20x20 grid)



6.3.2.4 Ising Model: Python Implementation

```
import numpy as np

def ising_gibbs(n, beta, n_steps=50000, seed=0):
    """Gibbs sampler for Ising model on n x n periodic grid."""
    np.random.seed(seed)
    config = np.random.choice([-1, 1], size=(n, n))

    for step in range(n_steps):
        i, j = np.random.randint(0, n, 2)
        # Sum of neighboring spins (periodic boundary)
        nbr_sum = (config[(i-1)%n, j] + config[(i+1)%n, j] +
                  config[i, (j-1)%n] + config[i, (j+1)%n])
        # Full conditional: P(spin = +1 | neighbors)
        h = beta * nbr_sum
        prob_plus = np.exp(h) / (np.exp(h) + np.exp(-h))
        config[i, j] = 1 if np.random.rand() < prob_plus else -1

    return config

# Run for beta = 0.44 (near critical temperature for 2D Ising)
config = ising_gibbs(n=20, beta=0.44, n_steps=100000)
magnetization = config.mean()
print(f"Magnetization: {magnetization:.4f}")
```

Critical temperature for 2D Ising: $\beta_c = \frac{1}{2} \ln(1 + \sqrt{2}) \approx 0.4407$.

6.4 Perfect Simulation: Motivation

Problem with standard MCMC: The chain (X_n) only *approximates* π — we never sample exactly from π .

Perfect Simulation

Generate a sample **exactly** from π in finite (random) time, without knowing when the chain has mixed.

Key idea: Coupling from the Past (CFTP) — run the chain *backwards* in time from $-\infty$ to 0, starting from **all** possible initial states simultaneously, and detect when they all coalesce.

Requires:

- A **monotone** update function φ (for efficient implementation).
- The ability to track only the **minimum** and **maximum** configurations.

6.4.1 Coupling from the Past (CFTP) I

State space: $\mathcal{S} = C^V$ with partial order $\preceq$ and update function φ .

Algorithm 50 (CFTP for Monotone Systems)

Require: State space $\mathcal{S} = C^V$, monotone π .

- 1 Set $n = 1$.
- 2 Start a bivariate chain $Y_k^{-N_n} = (X_k^{\xi_{\min}, -N_n}, X_k^{\xi_{\max}, -N_n})$ at time $-N_n$ from $(\xi_{\min}, \xi_{\max})$. Run using the **same** i.i.d. U_i random variables.
- 3 **If** at time 0 we have $Y_0^{-N_n} = (\xi, \xi)$: **return** ξ and **stop**.
- 4 **Else** set $n = n + 1$ and go to Step 2 (reuse previously generated U_i).

Theorem 6.4.2

- (i) Algorithm 50 terminates with probability 1.
- (ii) The output ξ has distribution exactly π .

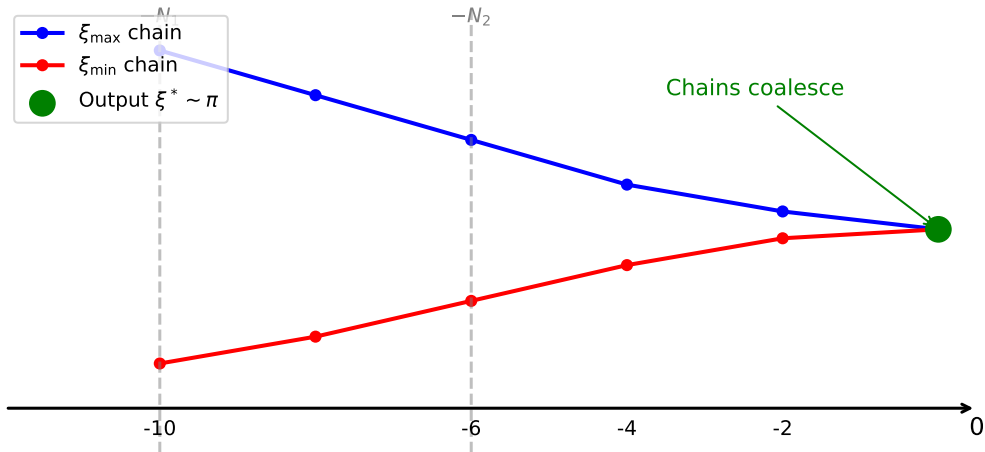
6.4.1 Coupling from the Past (CFTP) II

Key CFTP property (coupling inequality):

$$d_{\text{TV}}(\pi, \pi') \leq \mathbb{P}(Y_0^{-N_n} \neq Y_0^{\tau_{\text{cftp}}}) \leq \mathbb{P}(\tau_{\text{cftp}} < -N_n) \leq \varepsilon.$$

6.4.1 Coupling from the Past (CFTP) III

Coupling from the Past (CFTP): schematic



6.4.1 Coupling from the Past (CFTP) IV

Intuition:

- Two chains (from $\xi_{\min}$ and $\xi_{\max}$) coalesce because the update function is **monotone**:
 $\xi \preceq \xi' \Rightarrow \varphi(\xi, v, u) \preceq \varphi(\xi', v, u)$.
- Only need to track the min/max chains — all others lie between them.

6.4.2 Theory of CFTP

Definition: Monotone Markov Chain (Def. 6.4.5)

A Markov chain with update function φ is **monotone** w.r.t. $\preceq$ if

$$\xi \preceq \xi' \Rightarrow \varphi(\xi, v, u) \preceq \varphi(\xi', v, u) \quad \forall v \in V, u \in [0, 1].$$

Definition: Anti-monotone Chain (Def. 6.4.11)

The chain is **anti-monotone** if

$$\xi \preceq \xi' \Rightarrow \varphi(\xi', v, u) \preceq \varphi(\xi, v, u) \quad \forall v, u.$$

Examples:

- **Hard-core model** ($\lambda > 0$): monotone chain — higher occupation encourages more occupation.
- **Ferromagnet Ising** ($\beta > 0$): monotone chain.
- **Anti-ferromagnet Ising** ($\beta < 0$): anti-monotone chain — alternating update handles this.

6.4.1 CFTP: Python Sketch (Hard-core Model)

```
import numpy as np

def cftp_hardcore(G, lam, max_attempts=20):
    """
    CFTP for hard-core model on graph G with parameter lambda.
    G : dict {vertex: [neighbors]}
    """
    vertices = list(G.keys())
    n = len(vertices)

    for attempt in range(max_attempts):
        N = 2 ** attempt          # doubling strategy
        # Pre-generate uniform random variables U[i], V[i]
        Us = np.random.rand(N)
        Vs = np.random.randint(0, n, N)

        # Track min (all 0) and max (all 1 feasible)
        xi_min = {v: 0 for v in vertices}
        xi_max = {v: 1 for v in vertices}

        for k in range(N):
            v = vertices[Vs[k]]
            u = Us[k]
            # Update xi_min
            nbrs_min = any(xi_min[w] == 1 for w in G[v])
            xi_min[v] = 0 if nbrs_min else (1 if u < lam/(1+lam) else 0)
            # Update xi_max
```

6.4.4 Anti-ferromagnet Ising Model

Anti-ferromagnet Ising: $\beta < 0$

The update function:

$$\varphi(\xi, v, u) = \begin{cases} \xi(v \rightarrow -1) & \text{if } u \in [0, \alpha(\xi, v)), \\ \xi(v \rightarrow +1) & \text{if } u \in [\alpha(\xi, v), 1]. \end{cases}$$

For $\beta < 0$: $0 < \alpha(\xi, v) < 1$, and the chain is **anti-monotone**.

Algorithm 52 (CFTP for Anti-ferromagnet Ising)

Require: Graph $G = (V, E)$, parameter $\beta < 0$.

- ➊ Start bivariate chain $Y_{-N_n}^{-N_n} = (\xi_{\min}, \xi_{\max})$ where $\xi_{\min}$ assigns -1 to all vertices and $\xi_{\max}$ assigns $+1$ to all.
- ➋ Run using **same** U_k, W_k (vertex selector).
- ➌ The bivariate update interchanges the roles of the two chains when updating, exploiting anti-monotonicity.
- ➍ Stop when coalescence is detected.

Proposition 6.4.17: Algorithm 51 terminates with probability 1 and its output has distribution π .

6.5 Approximate Counting: Introduction I

Question: How many proper colorings does a graph have? How many independent sets?

Counting Problem

Given a graph $G = (V, E)$ and value set $C = \{1, \dots, r\}$, let

$$z_G = |\Lambda|, \quad \Lambda \subseteq C^V = \text{set of feasible configurations.}$$

Computing z_G **exactly** is #P-complete in general.

(ε, δ) -Approximation Scheme

An estimator $\hat{z}_G$ is an (ε, δ) -**approximation scheme** (FPRASC) for z_G if

$$\mathbb{P}(|\hat{z}_G - z_G| > \varepsilon z_G) \leq \delta,$$

with running time polynomial in $1/\varepsilon$, $\ln(1/\delta)$, and problem size.

6.5 Approximate Counting: Introduction II

Naïve estimator: Sample R random subsets of V , check if independent. With $p = z_G/2^M$,

$$\hat{z}_G^{\text{naive}} = 2^M \hat{Y}_R, \quad \mathbb{E} \hat{Y}_R = z_G/2^M.$$

Problem: p may be exponentially small, requiring exponentially many samples.

Solution (telescoping): Decompose

$$z_G = z_0 \times \frac{z_1}{z_0} \times \frac{z_2}{z_1} \times \cdots \times \frac{z_m}{z_{m-1}},$$

where each ratio $\varrho_i = z_i/z_{i-1} \geq 1/2$ is efficiently estimable.

6.5 Approximate Counting: Introduction III

$$z_G = z_0 \times \frac{z_1}{z_0} \times \frac{z_2}{z_1} \times \cdots \times \frac{z_m}{z_{m-1}}$$

$$= 2^M \cdot \varrho_1 \cdot \varrho_2 \cdots \varrho_m$$

where $\varrho_i = z_i/z_{i-1} \geq 1/2$ ensures efficient estimation

6.5.1 Independent Sets: FPRAS I

Setup: $G = (V, E)$, $M = |V|$, $m = |E|$. Let $G_i = (V, E_i)$ where $E_i = \{e_1, \dots, e_i\}$. Let $z_i = |\Lambda_{G_i}|$ (number of independent sets of G_i), and $z_0 = 2^M$.

Key Lemma 6.5.7

$\varrho_i = z_i/z_{i-1} \geq 1/2$ for all i .

Proof sketch: Adding edge (w, v) removes independent sets containing both w and v ; at most half of all independent sets of G_{i-1} contain both endpoints.

Proposition 6.5.4 (FPRAS for z_G)

Let (X_n) be rapidly mixing with stationary distribution $\pi_\xi^{(i)} = \mathbf{1}(\xi \in \Lambda_{G_i})/z_i$ and mixing time $\tau(\kappa)$. Then there exists an (ε, δ) -FPRASC for z_G with running time

$$\frac{200m^3}{\varepsilon^2} \ln\left(\frac{2m}{\delta}\right) \tau\left(\frac{\varepsilon}{16m}\right).$$

6.5.1 Independent Sets: FPRAS II

Algorithm 54: $(\varepsilon/2m, \delta/m)$ -FPRASC for ϱ_i

Require: Graphs G_{i-1}, G_i ; parameters δ, ε, m ; rapidly mixing chain $(X_n^{(i)})$.

- ① For $j = 1$ to $R = \frac{200m^2}{\varepsilon^2} \ln\left(\frac{2m}{\delta}\right)$:
 - Generate approximate sample ξ from $\pi^{(i-1)}$ (run chain for $\tau(\varepsilon/(16m))$ steps).
 - Set $\tilde{Y}_j^{(i)} = 1$ if $\xi \in \Lambda_{G_i}$, else 0.
- ② Return $\tilde{\varrho}_i = \frac{1}{R} \sum_{j=1}^R \tilde{Y}_j^{(i)}$.

Algorithm 55: (ε, δ) -FPRASC for z_G

- ① For $i = 1$ to m : estimate ϱ_i via Algorithm 54.
- ② Return $\hat{z}_G = 2^M \prod_{i=1}^m \tilde{\varrho}_i$.

6.5.2 Graph Colorings and Knapsack FPRAS I

Generalized framework (Proposition 6.5.13): Apply the same telescoping idea to any counting problem admitting:

- ① A sequence $\Lambda_0, \Lambda_1, \dots, \Lambda_t \equiv \Lambda$ with t polynomial in $|G|$ and z_0 known.
- ② Rapidly mixing chains with uniform stationary distribution on Λ_i and mixing time $\tau(\kappa)$.
- ③ $\varrho_i := z_i/z_{i-1} \geq 1/2$.

Running time: $\frac{200t^3}{\varepsilon^2} \ln\left(\frac{2t}{\delta}\right) \tau\left(\frac{\varepsilon}{16t}\right)$.

6.5.2 Graph Colorings and Knapsack FPRAS II

Graph colorings:

Lemma 6.5.14

If $r > 2\Delta^2$, then $\varrho_i \geq 1/2$.

Lemma 6.5.16

If $r > 2\Delta^2$, there exists an (ε, δ) -FPRASC for the number of proper colorings with running time $O\left(\frac{m^3}{\varepsilon^2} \ln\left(\frac{2m}{\delta}\right) \frac{r}{r-2\Delta} M \ln\left(\frac{16mM}{\varepsilon}\right)\right)$.

Knapsack:

Lemma 6.5.18 (Dyer *et al.* [156])

The Gibbs sampler for 0-1 knapsack (Algorithm 57) is rapidly mixing with mixing time

$$\tau(\kappa) = O(d^{9/2+\kappa}).$$

6.5.2 Graph Colorings and Knapsack FPRAS III

This gives an (ε, δ) -FPRASC for the number of knapsack solutions with running time $O(d^{15/2+\varepsilon/(16d)} / \varepsilon^2 \cdot \ln(2d/\delta))$.

6.5 Approximate Counting: Python Sketch

```
import numpy as np

def count_independent_sets_fpras(G, eps=0.1, delta=0.05, tau_func=None):
    """
    FPRASC for number of independent sets using edge-by-edge telescoping.
    G          : list of edges [(u,v), ...]
    eps, delta: accuracy parameters
    tau_func   : mixing time function (use constant for demo)
    """
    vertices = list(set(v for e in G for v in e))
    M = len(vertices)
    m = len(G)
    z0 = 2 ** M

    R = int(200 * m**2 / eps**2 * np.log(2*m/delta)) + 1
    tau = tau_func(eps/(16*m)) if tau_func else 100  # mixing steps

    rho_estimates = []
    current_edges = []
    for edge in G:
        # Estimate  $\rho_i = z_i / z_{i-1}$ 
        count_feasible = 0
        for _ in range(R):
            # Sample from  $\pi^{i-1}$  via Gibbs (simplified)
            xi = {v: np.random.randint(2) for v in vertices}
            # Check if xi is feasible for  $G_i$  (with new edge)
            new_u, new_v = edge
```

6.6 Motif Finding Problem I

Biological context: Identify conserved sequence patterns (motifs) in DNA/protein sequences.

Setup

- k sequences, each of length n , over alphabet $\Sigma = \{A, C, G, T\}$ (or $d = |\Sigma|$).
- A **motif** is a segment of length w injected into each row at a starting position $\xi_i^\bullet \in \{1, \dots, n - w + 1\}$.
- **Background** letters are i.i.d. with distribution $\theta_0 = (\theta_{1,0}, \dots, \theta_{d,0})^T$.
- **Motif** letters at position j have distribution $\theta_j = (\theta_{1,j}, \dots, \theta_{d,j})^T$.
- The **position weight matrix** (PWM) is $\Theta = (\theta_1, \dots, \theta_w)$.

Problem 2

Given sequences $\mathbf{R} = (r_{ij})$, $1 \leq i \leq k$, $1 \leq j \leq n$, and parameters θ_j ($j = 0, \dots, w$), find the starting positions $\xi^\bullet = (\xi_1^\bullet, \dots, \xi_k^\bullet)$.

6.6 Motif Finding Problem II

Likelihood function: For configuration ξ , define

- Matrix $\mathbf{A}_\xi$: $k \times w$ matrix, row i = segment of $\mathbf{R}$ starting at ξ_i .
- Matrix $\mathbf{C}_\xi$: $d \times w$ count matrix, $\mathbf{C}_\xi(i, j) = \#\{l : \mathbf{A}_\xi(l, j) = i\}$.

The complete-data likelihood is

$$f(\xi) = h(\xi) \times g(\xi) = \prod_{i=1}^d \theta_{i,0}^{\mathbf{D}_\xi(i)} \times \prod_{j=1}^w \prod_{i=1}^d \theta_{i,j}^{\mathbf{C}_\xi(i,j)}.$$

Energy of configuration:

$$H(\xi) = -\log f(\xi).$$

Gibbs distribution:

$$\pi_\xi = \frac{1}{z(T)} \exp\left(\frac{1}{T} \log f(\xi)\right) = \frac{f(\xi)^{1/T}}{z(T)}.$$

For small T , π concentrates on low-energy (high-likelihood) configurations $\xi^\bullet$.

6.6 Gibbs Sampler for Motif Finding

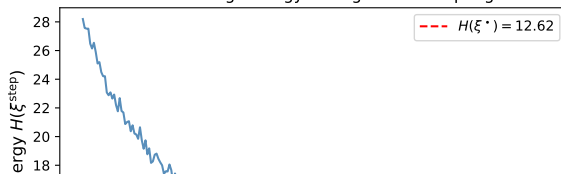
Gibbs Update (Algorithm 58)

- 1 Current configuration: $\xi = (\xi_1, \dots, \xi_k)$.
- 2 Pick row $v \in \{1, \dots, k\}$ uniformly at random.
- 3 Compute $\mathbf{A}_\xi$, $\mathbf{C}_\xi$, $\mathbf{R}_{[\xi]^c}$ (background rows excl. row v).
- 4 For each $c = 1, \dots, n - w + 1$, compute

$$p(c) = \frac{\pi_{\xi(v,c)}}{\sum_{r=1}^{n-w+1} \pi_{\xi(v,r)}}.$$

- 5 Sample $c \sim p(\cdot)$; set $X_{t+1} = \xi(v, c)$.

Motif finding: energy during Gibbs sampling



Numerical example (Sect. 6.6.3):

- $k = 30$ sequences, $n = 20$, $w = 13$.
- True positions ξ^* simulated.
- $H(\xi^*) = 12.62$.
- After 200 steps: $H(\xi^{200}) = 12.64$.

6.6 Motif Finding: Python Implementation

```
import numpy as np

def motif_gibbs(R, theta0, Theta, w, T=1.0, n_steps=200, seed=42):
    """
    Gibbs sampler for motif finding (Problem 2).
    R      : (k, n) integer matrix of observed sequences (0-indexed alphabet)
    theta0: background freq vector (length d)
    Theta  : (d, w) position weight matrix
    T      : temperature parameter
    """
    np.random.seed(seed)
    k, n = R.shape
    d = len(theta0)
    L = n - w + 1    # number of possible start positions

    def log_f(xi):
        log_lik = 0.0
        for i in range(k):
            s = xi[i]
            motif_part = R[i, s:s+w]
            for j, letter in enumerate(motif_part):
                log_lik += np.log(Theta[letter, j] + 1e-15)
        return log_lik / T

    xi = np.random.randint(0, L, k)    # initial configuration
    H_trace = []
```

6.7 Estimating $I = \mathbb{E}f(X)$: Setup

Goal: Estimate

$$I = \mathbb{E}f(X) = \sum_{s \in \mathcal{S}} f(s) \pi_s,$$

where $X \sim \pi$ and π is the stationary distribution of an ergodic Markov chain (X_n) .

MCMC Estimator

Run chain for R steps and use

$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R f(X_j).$$

Properties:

- **Not unbiased** if $X_0 \not\sim \pi$ (bias = $\beta/R + o(1/R)$).
- **Strongly consistent:** $\hat{Y}_R \rightarrow I$ a.s. as $R \rightarrow \infty$.
- **Asymptotically normal:** $\sqrt{R}(\hat{Y}_R - I) \xrightarrow{d} \mathcal{N}(0, \varsigma^2)$.

6.7.1 Ergodic Theorem and CLT for Markov Chains I

Proposition 6.7.1 (Strong Law / Ergodic Theorem)

If (X_n) is an ergodic finite Markov chain and f is any real-valued function, then

$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R f(X_j) \xrightarrow{R \rightarrow \infty} I = \sum_{s \in \mathcal{S}} \pi_s f(s) \quad \text{a.s.}$$

6.7.1 Ergodic Theorem and CLT for Markov Chains II

Proposition 6.7.5 (CLT for Markov Chains)

If (X_n) is an ergodic finite Markov chain and f is real-valued, then

$$\sqrt{R}(\hat{Y}_R - I) \xrightarrow{d} \mathcal{N}(0, \varsigma^2),$$

where the **asymptotic variance** (TAVC) satisfies

$$0 < \varsigma^2 = \lim_{R \rightarrow \infty} R \operatorname{Var} \hat{Y}_R < \infty.$$

The limits are independent of the initial distribution μ .

6.7.1 Ergodic Theorem and CLT for Markov Chains III

Regenerative structure: States of the chain returns to s_0 at times $\gamma_0, \gamma_1, \dots$. Cycle lengths $\eta_l = \gamma_l - \gamma_{l-1}$ and cycle sums $W_l = \sum_{j=\gamma_{l-1}}^{\gamma_l-1} f(X_j)$ are i.i.d.

$$I = \lim_{R \rightarrow \infty} \hat{Y}_R = \frac{\mathbb{E}W_1}{\mathbb{E}\eta_1} \quad (\text{ratio of i.i.d. means}).$$

Asymptotic variance formula:

$$\varsigma^2 = \frac{\text{Var}(W_1 - I\eta_1)}{\mathbb{E}\eta_1}.$$

Via fundamental matrix: Let $\mathbf{Z} = \mathbf{I} + \sum_{k=1}^{\infty} (\mathbf{P}^k - \mathbf{\Pi}) = (\mathbf{I} - (\mathbf{P} - \mathbf{\Pi}))^{-1}$. Then

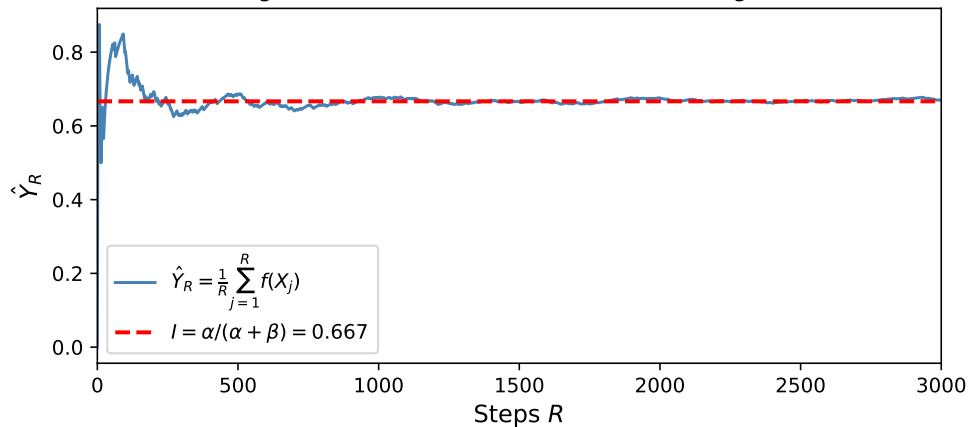
$$\varsigma^2 = \rho_0 + 2 \sum_{k=1}^{\infty} \rho_k = \text{Var} Y \left(1 + 2 \sum_{k=1}^{\infty} \rho_k \right),$$

and by Lemma 6.7.11:

$$\varsigma^2 = \rho_0 + 2\pi \text{diag}(\mathbf{f})(\mathbf{Z} - \mathbf{I})\mathbf{f}^T.$$

6.7.1 Ergodic Theorem and CLT for Markov Chains IV

Ergodic theorem: MCMC estimator convergence



6.7 Two-State Chain: Exact Asymptotic Variance

Example 6.7.12

Two-state chain $\mathcal{S} = \{0, 1\}$ with $\mathbf{P} = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}$, $f(x) = x$.

True value: $I = \alpha/(\alpha + \beta)$.

Asymptotic variance:

$$\zeta^2 = \frac{\alpha\beta(2 - \alpha - \beta)}{(\alpha + \beta)^3}.$$

Comparison with i.i.d.: $\text{Var}(Y^{\text{ind}}) = \alpha\beta/(\alpha + \beta)^2$, so

$$\frac{\zeta^2}{\text{Var}(Y^{\text{ind}})} = \frac{2 - \alpha - \beta}{\alpha + \beta}.$$

MC is better than i.i.d. when $\alpha + \beta > 1$.

Asymptotic variance ratio ($\beta = 0.25$)



6.7 Variance Estimation and Confidence Intervals

Problem: The asymptotic variance ς^2 is generally unknown. Several estimation methods:

Covariance Estimation (Eq. 6.7.11–6.7.12)

$$\hat{\rho}_k = \frac{1}{N-k} \sum_{j=1}^{N-k} (Y_j - \hat{Y}_N)(Y_{j+k} - \hat{Y}_N), \quad \hat{\varsigma}_N^2 = \sum_{k=-N}^N \hat{\rho}_k.$$

Method of Batch Means

Divide $Y_0, \dots, Y_{R-1}$ into m batches of size n . Batch means

$$\hat{Y}_k(n) = \frac{1}{n} \sum_{i=(k-1)n}^{kn-1} Y_i, \quad k = 1, \dots, m.$$

Confidence interval at level γ :

$$\hat{Y}_R \pm t_{m-1, 1-\gamma/2} \sqrt{\frac{V_m^2(n)}{m}}, \quad V_m^2(n) = \frac{1}{m-1} \sum_{j=1}^m (\hat{Y}_j(n) - \hat{Y}_R)^2.$$

6.7 MCMC Estimation: Python Implementation

```
import numpy as np
from scipy import stats

def mcmc_estimate_with_CI(chain_values, burn_in=500, n_batches=20):
    """
    Estimate  $I = E[f(X)]$  with 95% confidence interval via batch means.
    chain_values: array of  $f(X_j)$  values from MCMC chain
    """
    Y = chain_values[burn_in:]          # discard burn-in
    R = len(Y)
    I_hat = Y.mean()

    # Batch means
    batch_size = R // n_batches
    batches = [Y[k*batch_size:(k+1)*batch_size].mean() for k in range(n_batches)]
    batch_std = np.std(batches, ddof=1)

    # t-interval
    t_crit = stats.t.ppf(0.975, df=n_batches - 1)
    margin = t_crit * batch_std / np.sqrt(n_batches)

    # Asymptotic variance estimate via lag-k covariances
    N = 250    # truncation lag
    Y_c = Y - I_hat
    rho_hat = np.array([np.mean(Y_c[:R-k] * Y_c[k:]) for k in range(N+1)])
    zeta2_hat = rho_hat[0] + 2 * rho_hat[1:].sum()
```

6.8 Stable Simulation Scheme (Siegmund Duality) I

Goal: Estimate $\pi(A) = \sum_{s \in A} \pi_s$ for a **down-set** $A = \{s\}^\downarrow = \{s' \in \mathcal{S} : s' \preceq s\}$.

Setting: $\mathcal{S} = \{s_1, \dots, s_m\}$ partially ordered by $\preceq$; ergodic chain (X_n) with stationary π .

Definition 6.8.1 (Siegmund Dual)

The Markov chain (Z_n) with transition matrix $\mathbf{P}_Z$ is the **Siegmund dual** of (X_n) if

$$\forall (s_i, s_j) \in \mathcal{S}, \forall n \geq 0 : \quad \mathbf{P}_X^n(s_i, \{s_j\}^\downarrow) = \mathbf{P}_Z^n(s_j, \{s_i\}^\uparrow).$$

Taking $n \rightarrow \infty$:

$$\pi(\{s_j\}^\downarrow) = \mathbb{P}(Z \text{ is absorbed in } s_m \mid Z_0 = s_j).$$

6.8 Stable Simulation Scheme (Siegmund Duality) II

Key Formula (Eq. 6.8.2)

$$\pi(\{s_j\}^\downarrow) = \lim_{n \rightarrow \infty} \mathbf{P}_Z^n(s_j, \{s_i\}^\uparrow) = \mathbb{P}(Z \text{ absorbed at } s_m \mid Z_0 = s_j).$$

This relates the stationary distribution of (X_n) to **absorption probabilities** of (Z_n) .

Algorithm 59 (Stable Simulation Scheme)

Require: Möbius monotone chain (X_n) ; set $\{s\}^\downarrow$; R replications.

- 1 Compute Siegmund dual Z of X on $\mathcal{S}^* = \{s_0\} \cup \mathcal{S}$.
- 2 For $j = 1, \dots, R$: simulate $Z^{(j)}$ starting at s , until absorption.
- 3 Estimate

$$\hat{\pi}(\{s\}^\downarrow) = \frac{1}{R} \sum_{j=1}^R \mathbf{1}(Z^{(j)} \text{ absorbed at } s_m).$$

Required replications: $R \approx b^{-2}$ where b = desired half-width.

6.8 Stable Simulation Scheme (Siegmund Duality) III

Definition 6.8.1 (Möbius Monotonicity)

Chain (X_n) with t.m. $\mathbf{P}_X$ is **Möbius monotone** w.r.t. $\preceq$ if

$$\mathbf{C}^{-1}\mathbf{P}_X\mathbf{C} \geq 0,$$

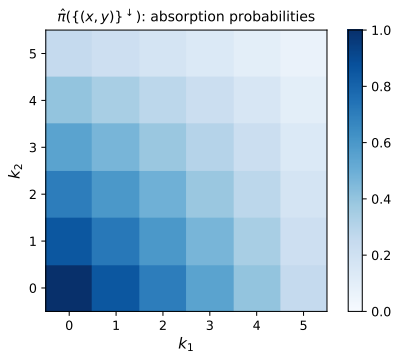
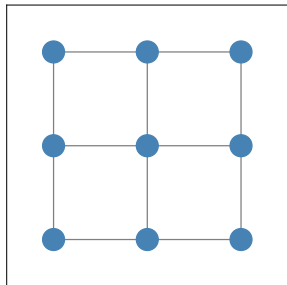
where $\mathbf{C}(s_i, s_j) = \mathbf{1}(s_i \preceq s_j)$ is the **ordering matrix** and $\mathbf{C}^{-1}$ is the **Möbius function**.

Lemma 6.8.2

There exists a Siegmund dual Z of X on $\mathcal{S}^* = \{s_0\} \cup \mathcal{S}$ if and only if (X_n) is Möbius monotone w.r.t. $\preceq$.

6.8 Stable Simulation Scheme (Siegmund Duality) IV

Partial order on $S = \{0, 1, 2\}^2$



6.8 Tandem Queue Example (Example 6.8.4)

Setup: Two-node closed tandem queue, $\mathcal{S} = \{0, \dots, N\}^2$, parameters $\lambda_1, \lambda_2, \mu_1, \mu_2$ with $\lambda_1 + \mu_1 + \lambda_2 + \mu_2 = 1$.

Goal: Estimate $\pi(\{(x, y)\}^\downarrow) = \pi(\{(x', y') : x' \leq x, y' \leq y\})$ for all $(x, y) \in \mathcal{S}$.

Results ($N = 5$, $\lambda_1 = 3/16$, $\lambda_2 = 1/16$, $\mu_1 = \mu_2 = 6/16$, $R = 4 \cdot 10^4$)

$y \backslash x$	0	1	2	3	4	5
0	0.030	0.081	0.145	0.236	0.351	0.502
1	0.058	0.139	0.249	0.381	0.550	0.763
2	0.079	0.181	0.310	0.468	0.652	0.884
3	0.094	0.207	0.353	0.511	0.710	0.946
4	0.102	0.224	0.372	0.540	0.743	0.981
5	0.111	0.235	0.383	0.558	0.762	1.000

Final 95% CI for $\hat{Y}_R$ (starting from $k_0 = 500$, $N = 250$): (23.15, 25.59).

6.9 Bibliographical Notes (Summary)

- **Markov chains:** Classical references include Norris [165], Meyn & Tweedie [147].
- **Metropolis algorithm:** Metropolis *et al.* [145] (1953); generalized by Hastings [78] (1970).
- **Gibbs sampler:** Introduced by Geman & Geman [62] for image restoration; popularized in statistics by Gelfand & Smith [61].
- **CFTP:** Propp & Wilson [170] (1996); monotone CFTP by Huber [85].
- **Approximate counting / FPRAS:** Jerrum, Valiant & Vazirani [100]; Dyer *et al.* [43].
- **Motif finding:** Lawrence *et al.* [119]; Gibbs approach by Liu *et al.* [128].
- **Ergodic theorem / CLT for MC:** Meyn & Tweedie [147]; Glynn & Iglehart [64].
- **Batch means:** Damerdji [30]; Alexopoulos & Seila [4].
- **Siegmund duality / stable simulation:** Siegmund [191]; Lorek & Szekli [130].

6.10 Selected Exercises I

Exercise 6.T.1 (Stationary distribution)

Find the stationary distribution of the Markov chain with transition matrix

$$\mathbf{P} = \begin{pmatrix} r_0 & p_0 & 0 & \cdots & 0 & q_0 \\ q_1 & r_1 & p_1 & \cdots & 0 & 0 \\ \vdots & & \ddots & & & \vdots \\ p_N & \cdots & 0 & q_N & r_N & \end{pmatrix}.$$

Hint: Use the detailed balance condition.

Exercise 6.T.8 (Total variation distance)

Show that $d_{\text{TV}}(\mu, \nu) = \max_{A \subseteq \mathcal{S}} |\mu(A) - \nu(A)|$.

6.10 Selected Exercises II

Exercise 6.T.12 (Two-state weather chain)

Chain on $\mathcal{S} = \{\text{rainy}, \text{sunny}\}$ with $\mathbf{P} = \begin{pmatrix} 1/2 & 1/2 \\ 1/10 & 9/10 \end{pmatrix}$.

- (a) Find the stationary distribution.
- (b) Starting from “rainy”, compute $d_{\text{TV}}(\mu\mathbf{P}^n, \pi)$ for $n = 1, 2, 3$.

Exercise 6.T.35 (Median trick for boosting)

Show that if $p > 1/2$ and $S_R = \sum_{i=1}^R Y_i$ where $Y_i \sim \text{Bernoulli}(p)$, then

$$\mathbb{P}(S_R > R/2) \geq 1 - \delta \quad \text{for } R = \frac{\log(1/\delta) \cdot 2p}{(p - 1/2)^2}.$$

(This is used in the FPRAS boosting lemma.)

Chapter 6 Summary

Key Algorithms:

- **Metropolis-Hastings** (Alg. 42–43): accept/reject with ratio $\pi_{s'}/\pi_s$.
- **Gibbs sampler** (Alg. 44–48): sample one coordinate from full conditional.
- **CFTP** (Alg. 50–53): perfect simulation via monotone coupling.
- **FPRAS** (Alg. 54–55): approximate counting via telescoping ratios.
- **Stable simulation** (Alg. 59): Siegmund duality for $\pi(A)$.

Key Theory:

- Detailed balance $\Rightarrow$ stationary π .
- Ergodic theorem: $\hat{Y}_R \rightarrow I$ a.s.
- CLT: $\sqrt{R}(\hat{Y}_R - I) \rightarrow \mathcal{N}(0, \varsigma^2)$.
- TAVC ς^2 : estimate via covariances or batch means.
- FPRAS requires $\varrho_i \geq 1/2$ and rapid mixing.
- CFTP: monotone $\Rightarrow$ coalescence $\Rightarrow$ exact sample from π .

The MCMC paradigm: Build a chain with stationary $\pi \rightarrow$ run it $\rightarrow$ average.